# EXPERIMENT 1

## Design and configure the hub network using cisco packet tracer

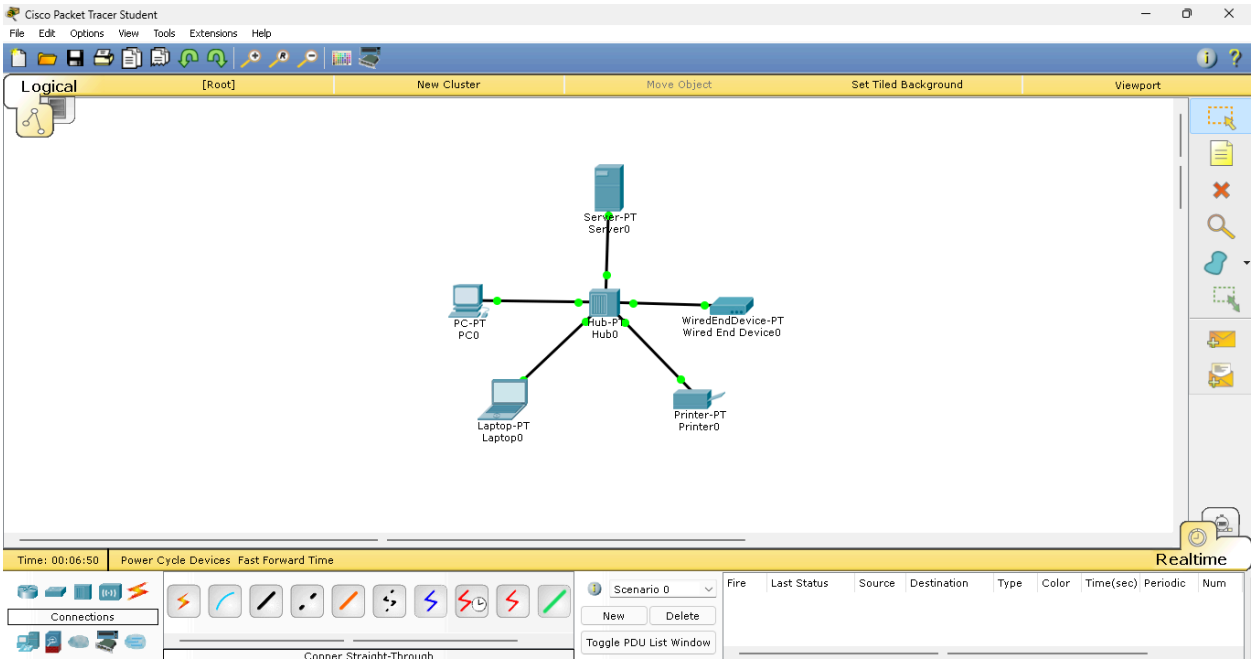
**AIM:**

To design and configure a Local Area Network (LAN) using a generic Cisco Switch (Switch-PT) and five Personal Computers (PC0–PC4), encompassing physical cabling, hardware module expansion, and logical Class A IP addressing.

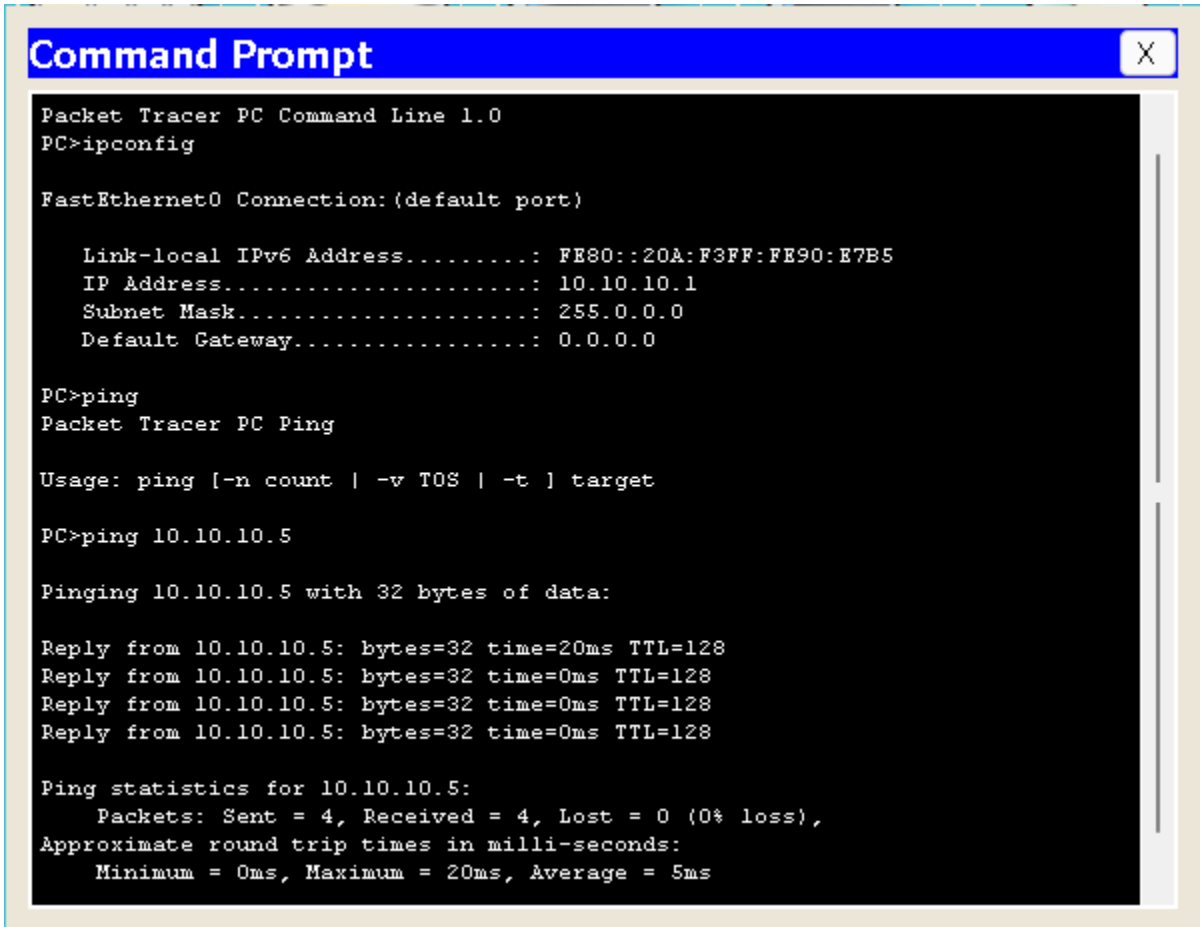
**PROCEDURE:**

1. Launch the Cisco Packet Tracer application.
2. From the Network Devices section, select Hubs .Drag and drop one Hub into the workspace.
   From End Devices, drag and drop five PCs (PC0 to PC4).
3. Connect the devices using Copper Straight-Through cable.Connect each PC's FastEthernet0 port to the Hub's available ports.
4. Assign IP Addresses.
5. Repeat the above steps for all PCs:
6. Verify Connectivity by opening Command Prompt on any PC.Use the ping command to test communication with other PCs.
7. Observe successful replies.

**BLOCK DIAGRAM:**



**OUTPUT:**




**RESULT:**

A network consisting of one Hub and five PCs was successfully designed and configured in Cisco Packet Tracer using Class A IP addressing. All PCs were able to communicate with each other, confirming proper network connectivity.

## Aim:

To design and simulate a basic network in Cisco Packet Tracer by connecting two hubs using a cross copper cable, where each hub is connected to four PCs using straight-through copper cables.

## Procedure:

1. Open **Cisco Packet Tracer**.

2. From the Network section, select Hub and place two hubs on the workspace.

3. From the End Devices section, select PC and place four PCs near each hub (total 8 PCs).

4. Connect each PC to its respective hub using Copper Straight-Through cable.

5. Select the cable → click on PC → choose Fast Ethernet → click on Hub → choose a port.

6. Connect the two hubs together using a Copper Cross-Over cable.

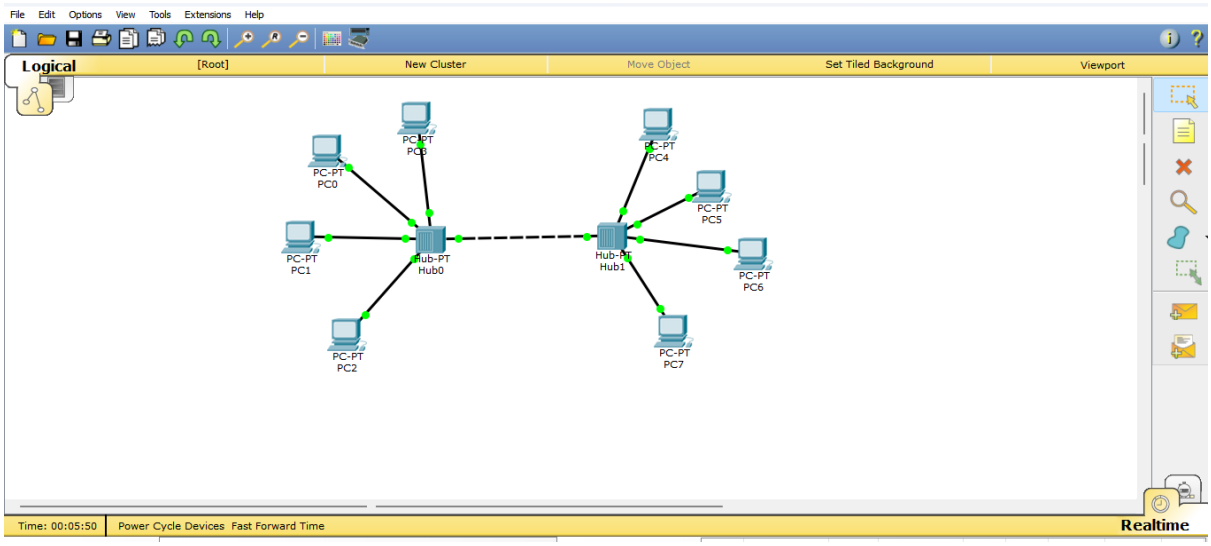
7. Click on the first hub → select a port → click on the second hub → select a port.

8. Assign IP addresses to all PCs in the same network

## Observation:

- All PCs were successfully connected to their respective hubs using straight-through copper cables.
- The two hubs were properly connected using a cross copper cable.
- Link lights on all devices indicated successful physical connections.
- Since hubs work at the physical layer, data packets were broadcast to all connected ports.
- When a packet was sent from one PC, all other PCs received the signal, but only the destination PC accepted the data.

**OUTPUT:**


File Edit Options View Tools Extensions Help
Logical
[Root]
New Cluster
Move Object
Set Tiled Background
Viewport
PC-PT PC0
PC-PT PC1
PC-PT PC2
PC-PT PC3
Hub-PT Hub0
Hub-PT Hub1
PC-PT PC4
PC-PT PC5
PC-PT PC6
PC-PT PC7
Time: 00:05:50
Power Cycle Devices Fast Forward Time
Realtime

EXPERIMENT - 3

**Design a Virtual LAN for LAB 1.a AND LAB 2.b**

**AIM :**

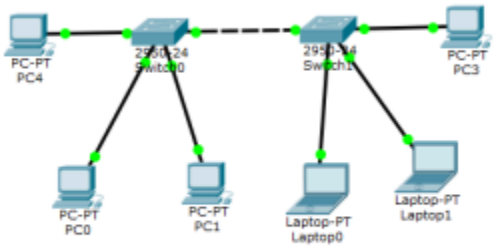
To design and configure a Virtual Local Area Network (VLAN) for AIML and CSD departments located in Block-A and Block-B using switches, such that faculty members of the same department are logically grouped in the same LAN while maintaining separate broadcast domains, and to verify connectivity using PDU packets in Cisco Packet Tracer.

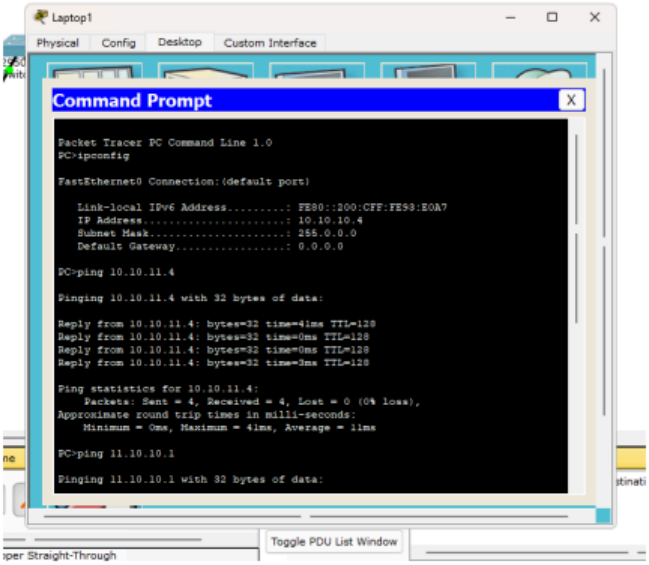
**PROCEDURE :**

1. Open Packet Tracer: Start the software and clear your workspace.
2. Add Switches: Place two switches and name them Block-A and Block-B.
3. Add PCs: Place two PCs at each switch for AIML and CSD faculty.
4. Connect PCs: Use Straight-Through cables to connect each PC to its switch.
5. Connect Switches: Link Switch-A to Switch-B using a Cross-Over cable.
6. Create VLANs (Switch-A): Open the CLI and create VLAN 10 (AIML) and VLAN 20 (CSD).
7. Create VLANs (Switch-B): Repeat the same VLAN creation (10 and 20) on the second switch.
8. Assign Ports: Map the AIML PC ports to VLAN 10 and CSD PC ports to VLAN 20 on both switches.
9. Set Trunking: Configure the link between the two switches as a Trunk port.
10. Set IPs: Give each PC an IP address (PCs in the same VLAN must be in the same network).
11. Test Mode: Enter Simulation Mode and use the Add Simple PDU (envelope icon) tool.
12. Verify: * Success: AIML to AIML / CSD to CSD.

- Fail: AIML to CSD.

OUTPUT :




**Observation**

1. 1. VLAN 10 (AIML) and VLAN 20 (CSD) were created successfully on both switches.
2. 2. PCs in the same VLAN communicated successfully across Block-A and Block-B.
3. 3. The trunk link allowed VLAN traffic to pass between the two switches.

4. 4. PCs in different VLANs could not communicate with each other.
5. 5. This shows that VLANs create separate broadcast domains in the network.

**RESULT**

A LAN was successfully created using a Switch-PT and five PCs. All systems were assigned Class A IP addresses with the subnet mask 255.0.0.0, placing them in the same subnet. Communication between all PCs was verified successfully using the ping command.

# Aim

To study and analyze five important network protocols—TCP, UDP, ARP, ICMP, and HTTP—using Wireshark, and understand how they work in real network communication.

# Five Protocols

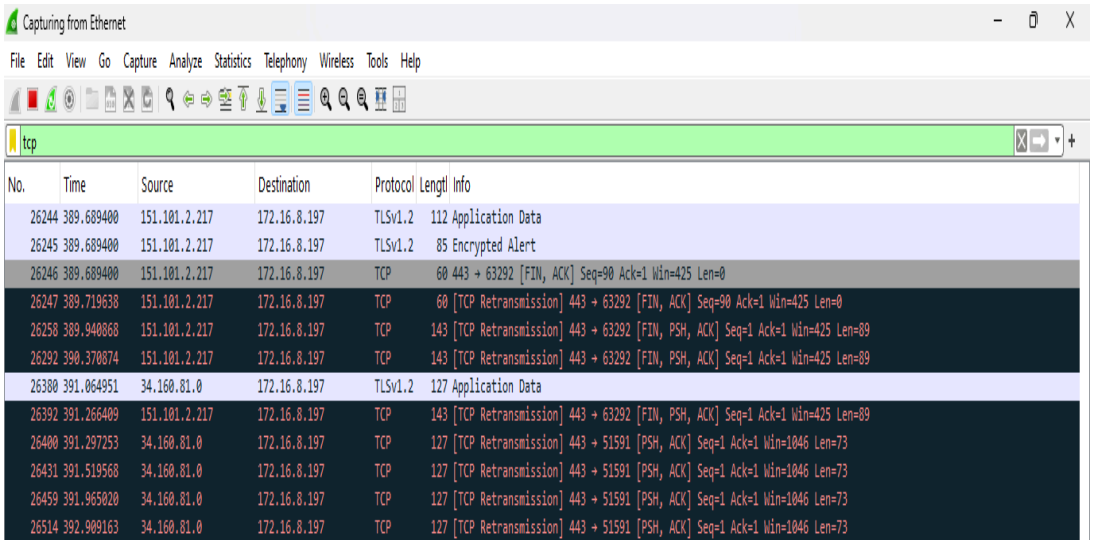
**1. TCP (Transmission Control Protocol)**
TCP is a reliable, connection-oriented protocol. It establishes a connection using a three-way handshake (SYN, SYN-ACK, ACK) and ensures safe and ordered data delivery.

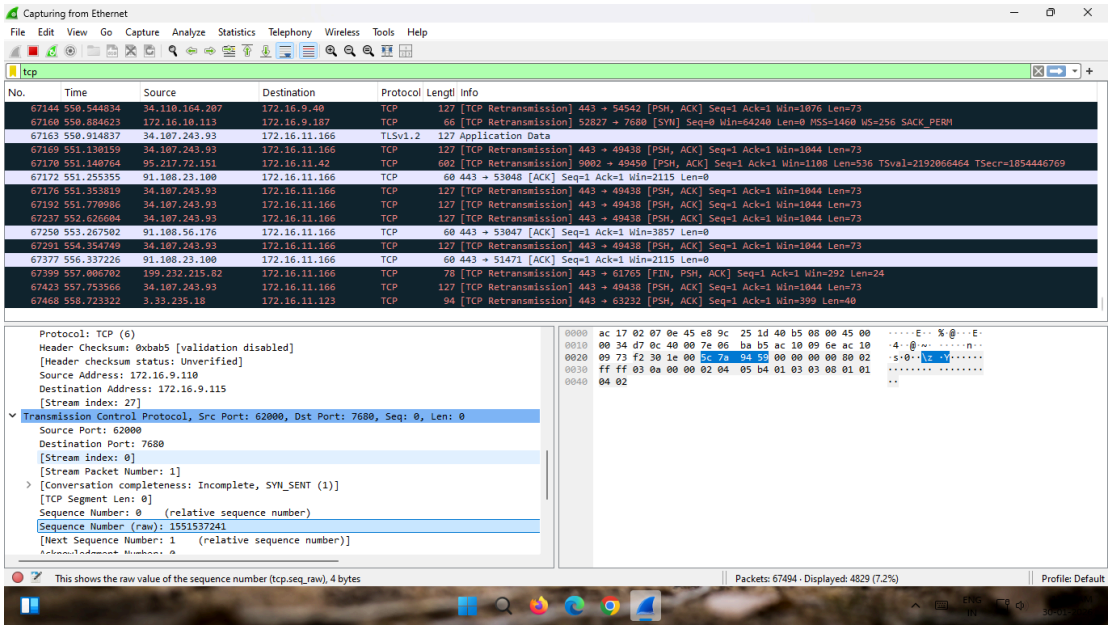
**2. UDP (User Datagram Protocol)**
UDP is a fast, connectionless protocol. It sends data without establishing a connection or waiting for acknowledgment.

**3. ARP (Address Resolution Protocol)**
ARP is used to find the MAC address of a device when its IP address is known. It sends a broadcast request and receives a reply.

**4. ICMP (Internet Control Message Protocol)**
ICMP is used for network testing and error messages. The ping command uses ICMP Echo Request and Echo Reply.

**5. HTTP (Hypertext Transfer Protocol)**
HTTP is an application layer protocol used to load websites. It works on a request–response model over TCP.




# Procedure

1. Open Wireshark and select the active network interface.
2. Start packet capture.
3. Generate traffic by browsing websites or using commands like `ping` and `nslookup`.
4. Stop capture after collecting enough packets.
5. Apply filters like `tcp`, `udp`, `arp`, `icmp`, and `http`.

6. Observe packet details and use Flow Graph to see communication flow.

## Observations

1. TCP showed a three-way handshake before sending data.
2. UDP sent packets without connection setup.
3. ARP showed broadcast request and reply for MAC address.
4. ICMP displayed Echo Request and Echo Reply.
5. HTTP showed GET request and 200 OK response.

## Result

The experiment was completed successfully. The working of TCP, UDP, ARP, ICMP, and HTTP was clearly understood using Wireshark packet capture and flow analysis.

# EXPERIMENT 5

## Implementing a Hamming Code sender program

**Aim:**

To design and implement a Python Sender Utility that encodes a binary data stream using Hamming Code (even parity) by inserting check bits at appropriate positions and generating the final bitstream for reliable transmission in a noisy environment.

**Procedure:**

1. Start the program and read the input binary data stream from the user.
2. Determine the number of data bits present in the input string.
3. Calculate the required number of check bits **r** using the condition 2r≥m+r+12^r \ge m + r + 12r≥m+r+1, where **m** is the number of data bits.
4. Identify the positions for check bits in the bitstream at powers of two such as 1, 2, 4, 8, etc.
5. Create a new bitstream with positions reserved for both data bits and check bits.
6. Insert the data bits into all positions that are not powers of two.
7. Assign each data bit to the appropriate check bit groups based on binary position indexing.
8. For each check bit, count the number of 1s in the positions it monitors.
9. Set the value of the check bit so that the total number of 1s in its group becomes even (even parity).
10. Place the calculated check bit values in their respective positions in the bitstream.
11. Combine all bits to form the final encoded Hamming code packet.
12. Display the final encoded bitstream as the output ready for transmission

**OUTPUT:**

```
Enter the data bits: 1011
Hamming Code: 1010101
Enter the received Hamming Code: 1010101
No error detected.
Corrected Hamming Code: 1010101
```

**RESULT:**

Hence successfully implemented to encode the given binary data using Hamming Code with Even Parity.

# EXPERIMENT 6

## Sliding Window

**Aim:**

To write and execute a C program to simulate the Sliding Window Protocol used in computer networks for flow control and to demonstrate how frames are transmitted and acknowledgements are received**.**

**Procedure:**

1. Start the program.
2. Declare variables for:
   - Window size (w)
   - Number of frames (f)
   - Frame array (frames[50])
   - Loop variable (i).
3. Prompt the user to enter the window size.
4. Prompt the user to enter the number of frames to be transmitted.
5. Read the frame numbers from the user and store them in an array.
6. Display a message explaining that frames will be transmitted using the Sliding Window Protocol assuming no errors.
7. Use a for loop to send frames one by one.
8. After sending frames equal to the window size, display a message that acknowledgement has been received.
9. Continue sending the next set of frames until all frames are transmitted.
10. If the remaining frames are less than the window size, send them and display the acknowledgement.
11. Stop the program.

**OUTPUT:**

```
Enter window size: 5

Enter number of frames to transmit: 4

Enter 4 frames: 1 2 3 4

With sliding window protocol the frames will be sent in the following manner
    (assuming no corruption of frames)

After sending 5 frames at each stage sender waits for acknowledgement sent
    by the receiver

1 2 3 4
Acknowledgement of above frames sent is received by sender
```

**RESULT:**

The C program to simulate the Sliding Window Protocol was successfully executed and the transmission of frames along with acknowledgements was displayed.

## EXPERIMENT – 7

# Subnetting and Connectivity Check

**AIM :** To design and configure two subnets from the network 192.168.10.0/25 and verify connectivity between computers in different subnets by sending PDU packets.

**PROCEDURE:**

Start **Cisco Packet Tracer**.

Create a new network topology.

From the device list, add the following components:

- 1 Switch for each subnet
- At least **3 PCs for each subnet** (total 6 PCs)
- 1 Router to connect the two subnets.

Connect the devices using **Copper Straight-Through cables**:

- PCs to their respective switches.
- Switches to the router interfaces.

Perform **subnetting** for the network **192.168.10.0/25**.

Network: **192.168.10.0/25**
Subnet Mask: **255.255.255.128**

Divide into **2 subnets**:

**Subnet 1**

- Network ID: **192.168.10.0**

- Range: **192.168.10.1 – 192.168.10.62 ●**

  Broadcast: **192.168.10.63**

**Subnet 2**

- Network ID: **192.168.10.64**
- Range: **192.168.10.65 – 192.168.10.126**
- Broadcast: **192.168.10.127**

Assign **IP addresses** to the PCs.

**Subnet 1 PCs**

- PC1 → 192.168.10.2 /25
- PC2 → 192.168.10.3 /25
- PC3 → 192.168.10.4 /25

**Subnet 2 PCs**

- PC4 → 192.168.10.66 /25
- PC5 → 192.168.10.67 /25
- PC6 → 192.168.10.68 /25

Configure **router interfaces**:

- Interface for Subnet 1 → 192.168.10.1 ●

Interface for Subnet 2 → 192.168.10.65

Set the **default gateway** on each PC:

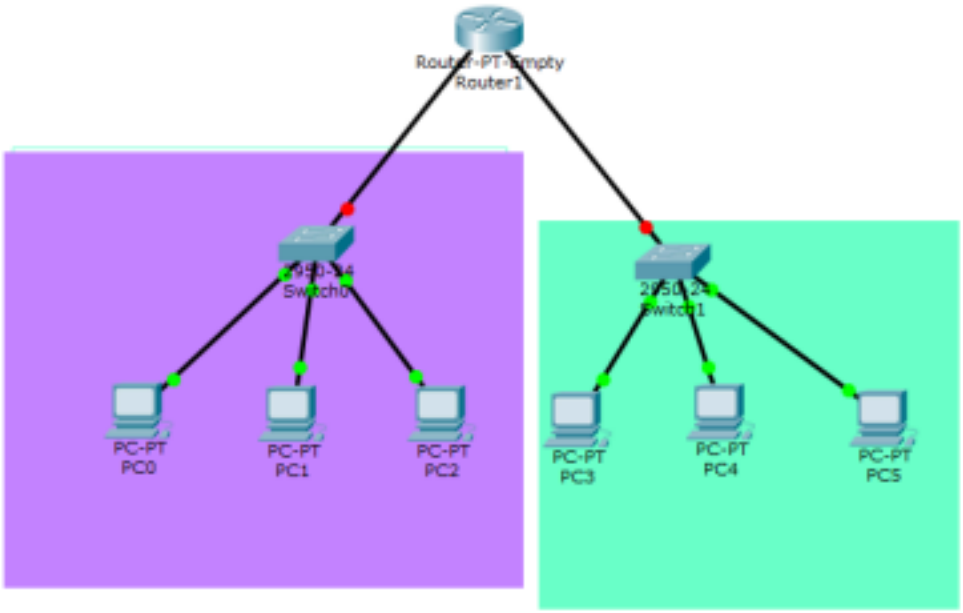
- Subnet 1 → 192.168.10.1
- Subnet 2 → 192.168.10.65

Switch to **Simulation Mode** in Packet Tracer.

Use the **Add Simple PDU tool** to send packets between PCs from different subnets.

Observe the **successful packet transmission** indicating connectivity.

## OUTPUT:




## RESULT :

Thus, the network **192.168.10.0/25** was successfully divided into **two subnets**, and connectivity between computers in different subnets was verified by sending **PDU packets using Cisco Packet Tracer**.